

HIGH-LEVEL SYNTHESIS OF QUANTUM CIRCUITS FROM C VIA MLIR AND QUANTUM-AWARE INTERMEDIATE REPRESENTATIONS

*Code Transformation and
Optimization*

AA 2024/2025

Leonardo Ignazio Pagliochini

10707169

Francesco Rosnati

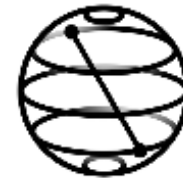
10562427



C code



MLIR

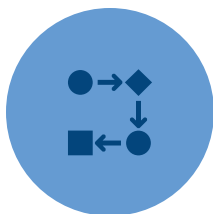


OpenQASM

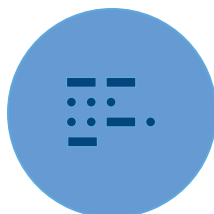
Table of contents



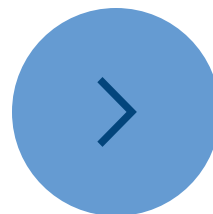
INTRODUCTION
AND GOALS



PIPELINE
DESCRIPTION



MLIR AND QMLIR



TRANSLATION OF C
LOGIC



PERFORMANCE
ANALYSIS



CONCLUSIONS AND
FUTURE
DEVELOPMENT



Introduction and goals

This work introduces a **compilation pipeline** that synthesizes fully reversible quantum circuits directly from classical C programs.

One of the main challenges is to translate the C logic to match the

Fundamental Quantum Constraints



Introduction and goals

This work introduces a **compilation pipeline** that synthesizes fully reversible quantum circuits directly from classical C programs.

One of the main challenges is to translate the C logic to match the

Fundamental Quantum Constraints

No Cloning

No Deleting

No Signaling

Unitarity and
Reversibility

Linearity



Introduction and goals

Challenges of NISQ programming

Together with the quantum constraints, the compiler should take care of the physical constraints of the available quantum architecture:



Introduction and goals

Challenges of NISQ programming

Together with the quantum constraints, the compiler should take care of the physical constraints of the available quantum architecture:

Resource
Efficiency

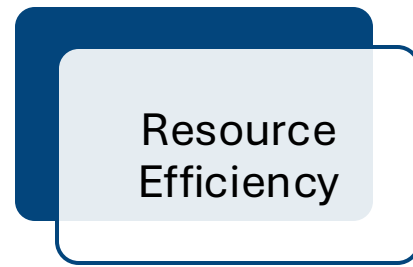
High-level
synthesis and
optimization



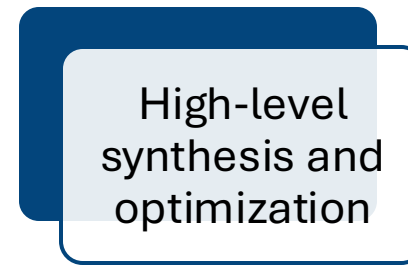
Introduction and goals

Challenges of NISQ programming

Together with the quantum constraints, the compiler should take care of the physical constraints of the available quantum architecture:



**QFT-Based
Operations**



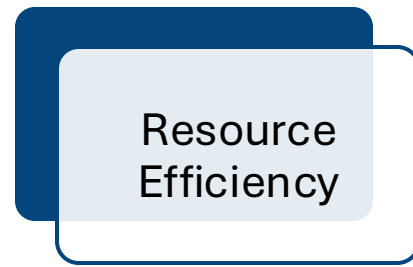
**Intermediate
representations**



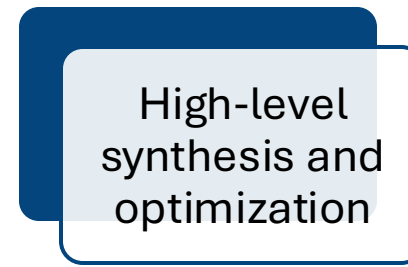
Introduction and goals

Challenges of NISQ programming

Together with the quantum constraints, the compiler should take care of the physical constraints of the available quantum architecture:



**QFT-Based
Operations**



**Intermediate
representations**

- ✓ Classical Optimizations
- ✓ Quantum Transpiling Optimizations



Introduction and goals

Why is addressing this challenge valuable for quantum technologies?



Introduction and goals

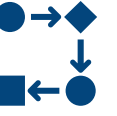
Why is addressing this challenge valuable for quantum technologies?



Because quantum computing is rapidly advancing, making the integration of classical code especially relevant

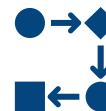


To reduce hybrid computing and shift classical routines entirely to quantum systems



Pipeline Description

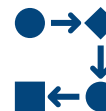
The compilation pipeline is designed as a modular **multi-stage process** that transforms high-level classical programs into executable quantum circuits. It progresses through a sequence of abstractions that gradually lower the source code while **preserving semantic structure** and **complying with quantum mechanical constraints**



Pipeline Description

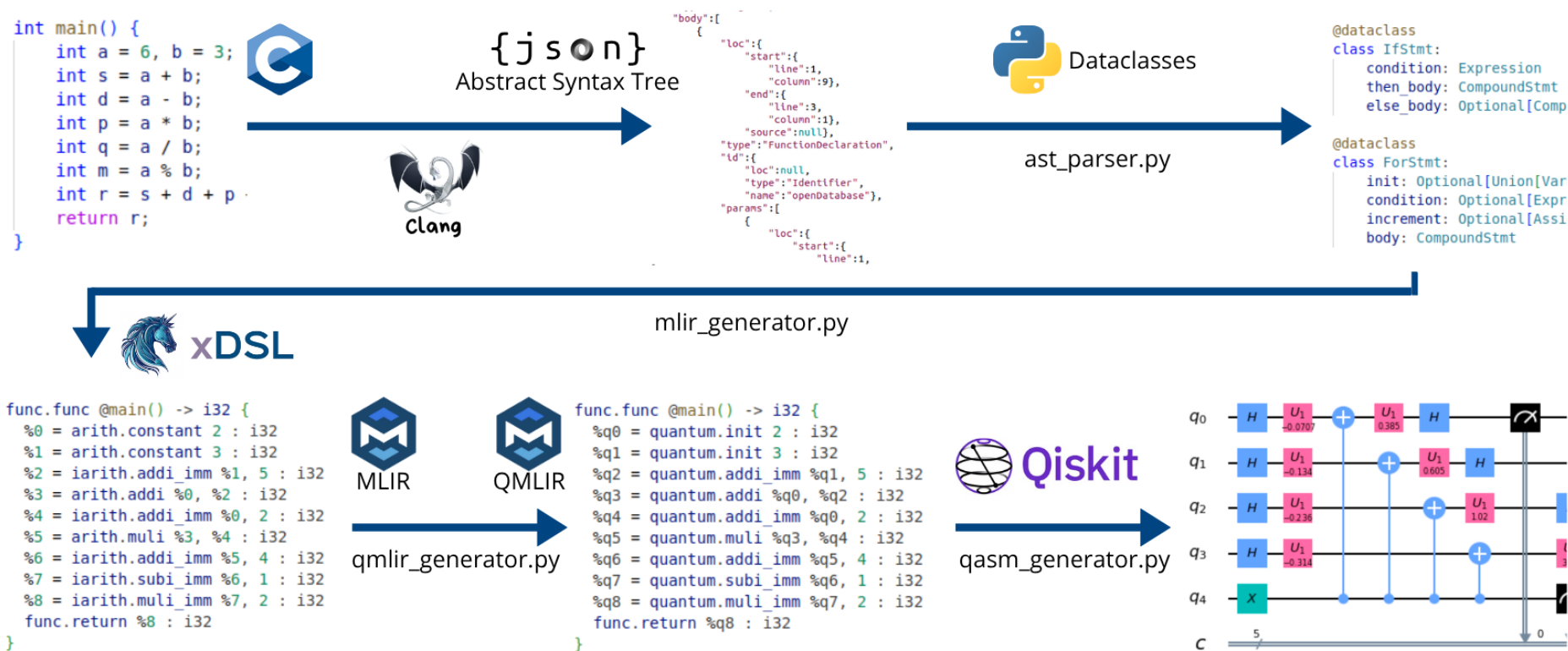
The compilation pipeline is designed as a modular **multi-stage process** that transforms high-level classical programs into executable quantum circuits. It progresses through a sequence of abstractions that gradually lower the source code while **preserving semantic structure** and **complying with quantum mechanical constraints**





Pipeline Description

Adopted Technologies





Custom MLIR Dialect for C-to-IR

We define a custom MLIR-based dialect to lower a subset of C programs into a structured intermediate representation.

The dialect combines:

- **Standard MLIR operations** (*arith, func, cf, builtin*)
- **Custom operations** (*iarith.*_imm*) for arithmetic with constants



Custom MLIR Dialect for C-to-IR

We define a custom MLIR-based dialect to lower a subset of C programs into a structured intermediate representation.

The dialect combines:

- **Standard MLIR operations** (*arith*, *func*, *cf*, *builtin*)
- **Custom operations** (*iarith.*_imm*) for arithmetic with constants

Type	MLIR Dialect	Example
Arithmetic	<i>arith.addi, subi, muli, divsi</i>	<code>%1 = arith.addi %a, %b</code>
Immediate arith.	<i>iarith.addi_imm, subi_imm, etc.</i>	<code>%1 = iarith.addi_imm %a, 5</code>
Comparison	<i>arith.cmpi</i>	<code>%cmp = cmpi slt %a, %b</code>
Constants	<i>arith.constant</i>	<code>%c = constant 42 : i32</code>
Function	<i>func.func, func.return</i>	<code>func.func @main() -> i32 { ... }</code>
Conditionals	<i>cf.cond_br</i>	<code>cond_br %cond, ^then, ^else</code>



Custom MLIR Dialect for C-to-IR

We define a custom MLIR-based dialect to lower a subset of C programs into a structured intermediate representation.

The dialect combines:

- **Standard MLIR operations** (*arith, func, cf, builtin*)
- **Custom operations** (*iarith.*_imm*) for arithmetic with constants

IF Statements

Translated using **cf.cond_br**

Logical && → nested if

Logical || → duplicated if blocks

FOR Loops

Translated by **loop unrolling** (up to MAX_UNROLL)

Each iteration includes:

- condition check
- body execution
- increment
- re-evaluation



Custom MLIR Dialect for C-to-IR

We define a custom MLIR-based dialect to lower a subset of C programs into a structured intermediate representation.

The dialect combines:

- **Standard MLIR operations** (*arith, func, cf, builtin*)
- **Custom operations** (*iarith.*_imm*) for arithmetic with constants

IF Statements

Translated using **cf.cond_br**

Logical && → nested if

Logical || → duplicated if blocks

FOR Loops

Translated by **loop unrolling** (up to MAX_UNROLL)

Each iteration includes:

- condition check
- body execution
- increment
- re-evaluation

Semantic Rules:

- Each variable must be **declared and initialized** before use
- No arrays, structs, or pointers (**scalar integers only**)
- SSA form **strictly enforced**



Quantum MLIR Dialect

A custom MLIR dialect for quantum arithmetic and control logic:

- Replaces standard *arith* operations with **quantum-safe equivalents**
- Enforces constraints such as **no copying**, **controlled execution**, and **SSA integrity**
- Supports both **classical-to-quantum translation** and **high-level synthesis**



Quantum MLIR Dialect

A custom MLIR dialect for quantum arithmetic and control logic:

- Replaces standard *arith* operations with **quantum-safe equivalents**
- Enforces constraints such as **no copying**, **controlled execution**, and **SSA integrity**
- Supports both **classical-to-quantum translation** and **high-level synthesis**

Quantum Arithmetic Operations

Operation	MLIR Syntax	Description
quantum.addi	<code>%r = quantum.addi %a, %b : i32</code>	Quantum integer addition
quantum.subi	<code>%r = quantum.subi %a, %b : i32</code>	Quantum subtraction
quantum.muli	<code>%r = quantum.muli %a, %b : i32</code>	Quantum multiplication
quantum.divsi	<code>%r = quantum.divsi %a, %b : i32</code>	Signed division



Quantum MLIR Dialect

A custom MLIR dialect for quantum arithmetic and control logic:

- Replaces standard *arith* operations with **quantum-safe equivalents**
- Enforces constraints such as **no copying**, **controlled execution**, and **SSA integrity**
- Supports both **classical-to-quantum translation** and **high-level synthesis**

Immediate Arithmetic

Operation	MLIR Syntax	Description
quantum.addi_imm	<code>%r = quantum.addi_imm %a, 5 : i32</code>	Add immediate to quantum register
quantum.subi_imm	<code>%r = quantum.subi_imm %a, 3 : i32</code>	Subtract immediate
quantum.muli_imm	<code>%r = quantum.muli_imm %a, 2 : i32</code>	Multiply by immediate
quantum.divsi_imm	<code>%r = quantum.divsi_imm %a, 4 : i32</code>	Divide by immediate



Quantum MLIR Dialect

A custom MLIR dialect for quantum arithmetic and control logic:

- Replaces standard *arith* operations with **quantum-safe equivalents**
- Enforces constraints such as **no copying**, **controlled execution**, and **SSA integrity**
- Supports both **classical-to-quantum translation** and **high-level synthesis**

Controlled Arithmetic Operations

Operation	MLIR Syntax	Description
quantum.c_addi	<code>%r = quantum.c_addi %a, %b, %ctrl : i32</code>	Addition executed only if ctrl is 1
quantum.c_addi_imm	<code>%r = quantum.c_addi_imm %a, 5, %ctrl : i32</code>	Immediate addition under control
(same for <i>subi</i> , <i>muli</i> , etc.)		



Quantum MLIR Dialect

A custom MLIR dialect for quantum arithmetic and control logic:

- Replaces standard *arith* operations with **quantum-safe equivalents**
- Enforces constraints such as **no copying**, **controlled execution**, and **SSA integrity**
- Supports both **classical-to-quantum translation** and **high-level synthesis**

Control Flow and Logic

Operation	MLIR Syntax	Description
quantum.cmpi	<code>%r = quantum.cmpi %a, %b [predicate] : i32</code>	Comparison between quantum registers (returns i1)
quantum.and	<code>%r = quantum.and %a, %b : i1</code>	Logical AND on control qubits
quantum.not	<code>%r = quantum.not %a : i1</code>	Logical NOT



Quantum MLIR Dialect

A custom MLIR dialect for quantum arithmetic and control logic:

- Replaces standard *arith* operations with **quantum-safe equivalents**
- Enforces constraints such as **no copying**, **controlled execution**, and **SSA integrity**
- Supports both **classical-to-quantum translation** and **high-level synthesis**

Register Initialization

Operation	MLIR Syntax	Description
quantum.init	<code>%r = quantum.init 7 : i32</code>	Initialize quantum register with value
quantum.c_init	<code>%r = quantum.c_init(%ctrl) 3 : i32</code>	Controlled initialization with ctrl



Translation of C logic: Integer Representation

The system allows scalar integer declarations with optional initialization and supports assignments to named variables. Each integer is represented in the quantum computer by a quantum register using two's complement representation. Initialization is performed by applying NOT gates to the qubits that need to be set to one.

```
int main()  
{  
    int a = 5;  
    return a;  
}
```




Translation of C logic: Integer Representation

The system allows scalar integer declarations with optional initialization and supports assignments to named variables. Each integer is represented in the quantum computer by a quantum register using two's complement representation. Initialization is performed by applying NOT gates to the qubits that need to be set to one.

```
int main()  
{  
    int a = 5;  
    return a;  
}
```

5 = 000101

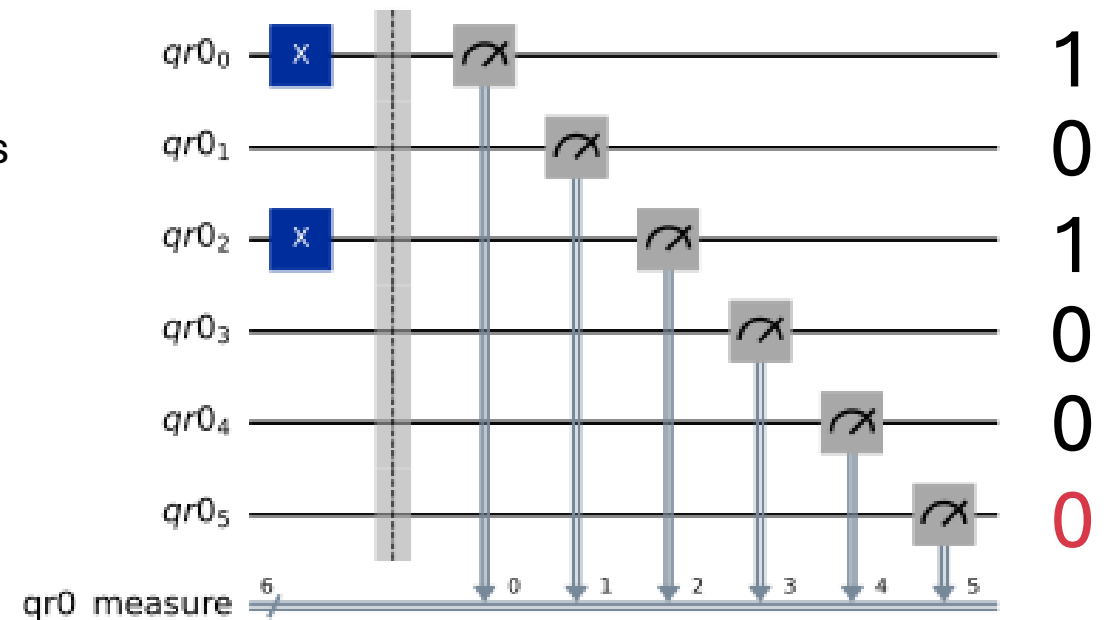


Translation of C logic: Integer Representation

The system allows scalar integer declarations with optional initialization and supports assignments to named variables. Each integer is represented in the quantum computer by a quantum register using two's complement representation. Initialization is performed by applying NOT gates to the qubits need to be set to one.

```
int main()  
{  
    int a = 5;  
    return a;  
}
```

5 = 000101





Translation of C logic: Operations

add_in_place : Adds two quantum registers, modifying the first register with the result

add : Adds two quantum registers, storing the result in a new register

addi_in_place : Adds a classical integer to a quantum register, modifying the input register

addi : Adds a classical integer to a quantum register, result stored in a new register

sub : Subtracts one quantum register from another, returns a new register with the result

subi : Subtracts a classical integer from a quantum register, returns a new register with the result

invert : Negates a quantum register in two's complement form (bitwise NOT + add 1)

abs_val : Converts a quantum register to its absolute value in place

mul : Multiplies two quantum registers, result stored in a new register

muli : Multiplies a quantum register by a classical constant, result stored in a new register

divu : Performs unsigned division between two quantum registers, returns quotient and remainder

divui : Performs unsigned division between a quantum register and a classical divisor, returns quotient and remainder

div : Performs signed division between two quantum registers, returns quotient and remainder

divi : Performs signed division between a quantum register and a classical divisor, returns quotient and remainder

equal : Compares two quantum registers for equality, returns a qubit storing the result

not_equal : Compares two quantum registers for inequality, returns a qubit storing the result

less_than : Compares whether one quantum register is less than another, returns a qubit storing the result

greater_than : Compares whether one quantum register is greater than another, returns a qubit storing the result

less_equal : Compares whether one quantum register is less than or equal to another, returns a qubit storing the result

greater_equal : Compares whether one quantum register is greater than or equal to another, returns a qubit storing the result

logical_and : Computes logical AND between two qubits, result stored in a new qubit

logical_or : Computes logical OR between two qubits, result stored in a new qubit

initialize_variable : Creates and initializes a quantum register with a classical integer value

initialize_bit : Initializes a single qubit to $|0\rangle$ or $|1\rangle$ based on a classical bit

measure : Measures a quantum register and stores results in a classical register

measure_single : Measures a single qubit and stores the result in a classical bit

simulate : Simulates the quantum circuit and interprets measured output in two's complement form

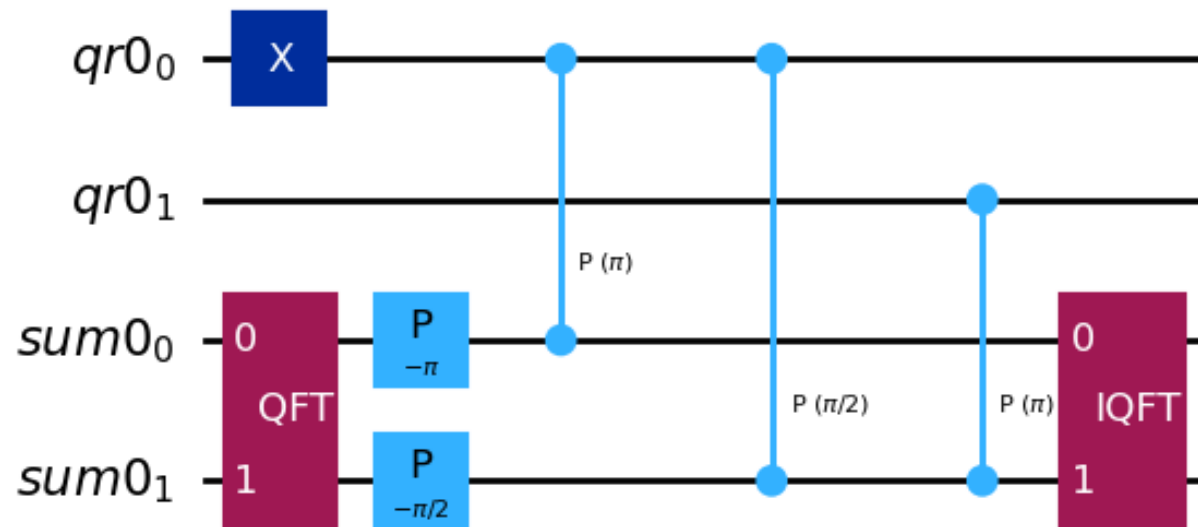
set_number_of_bits : Sets the global bitwidth used for all arithmetic operations

int_to_twos_complement : Converts a classical integer to its two's complement bit representation

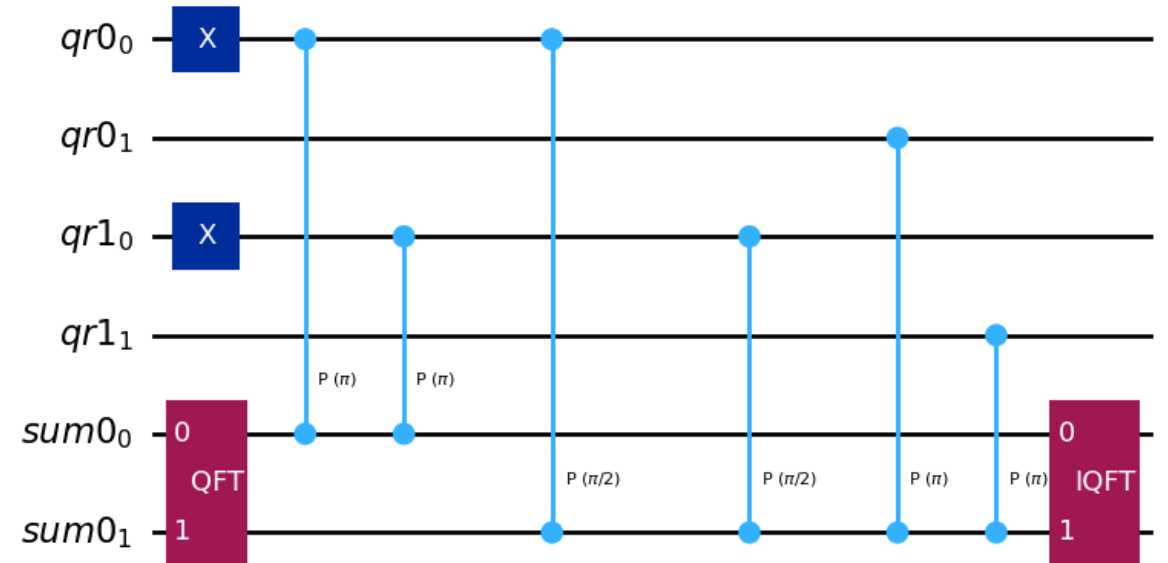
pad_register : Pads a quantum register with $|0\rangle$ qubits to reach a target size



Translation of C logic: Operations



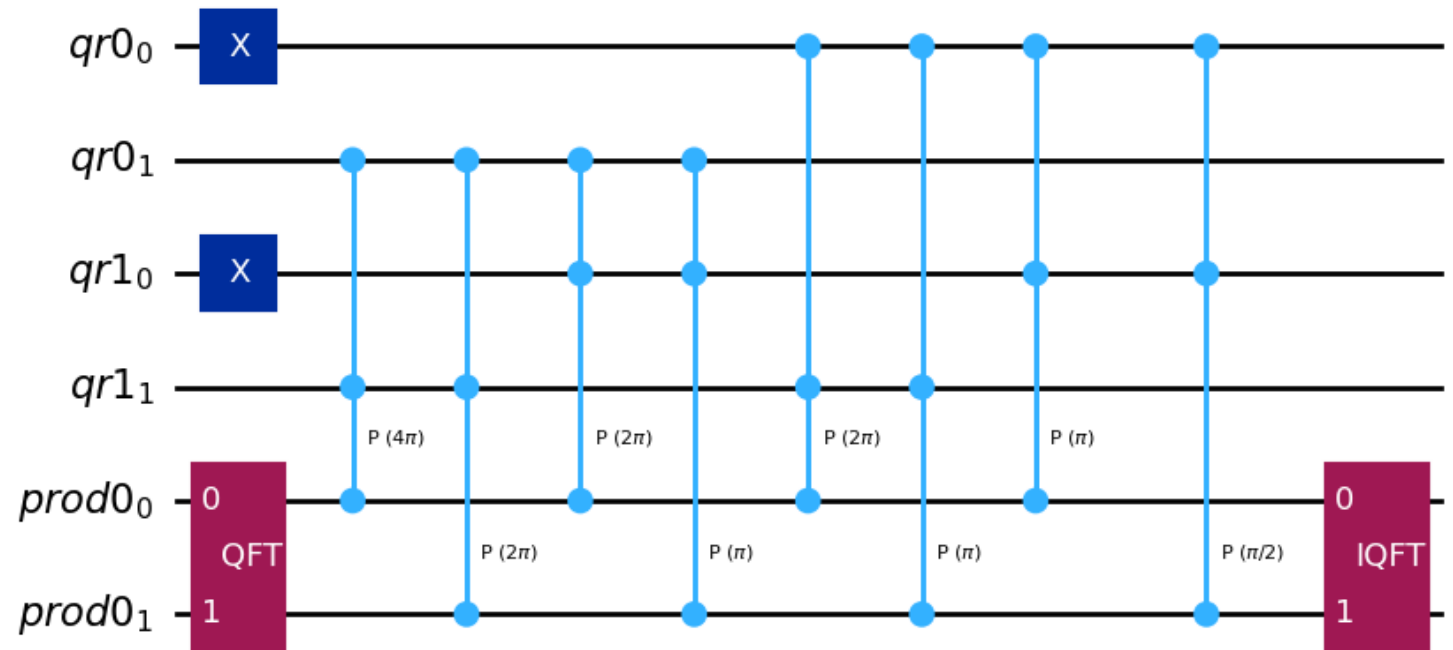
Add_imm operation



Add operation

Translation of C logic: Operations

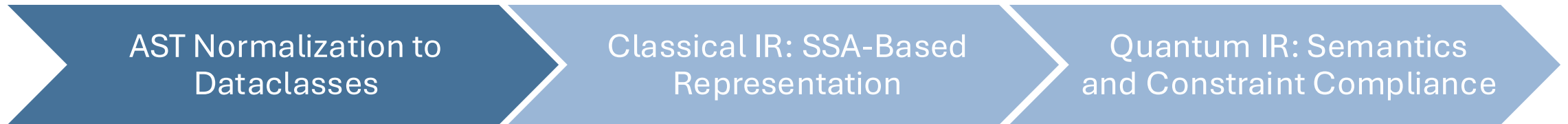
Multiplication





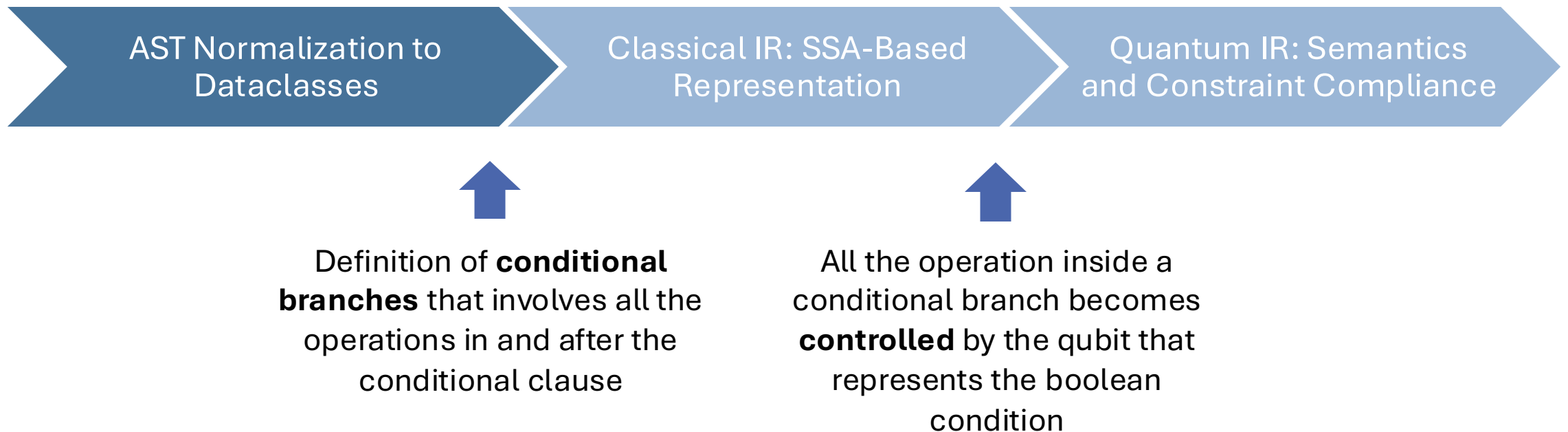
Translation of C logic: Conditional clauses

The implementation of the conditional clause is managed in two different phases of the pipeline:



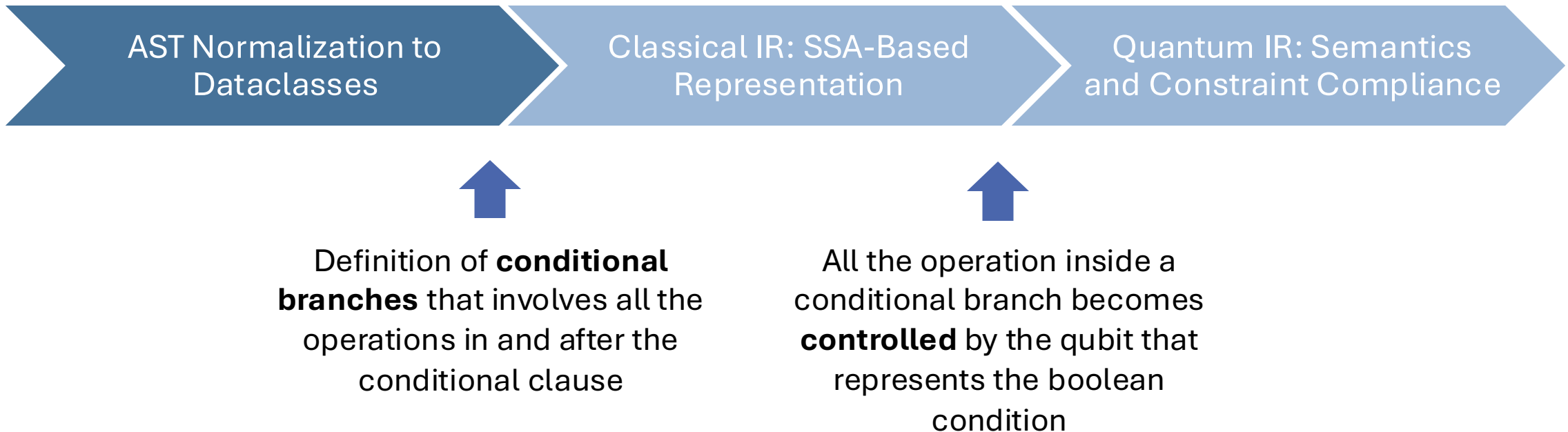
Translation of C logic: Conditional clauses

The implementation of the conditional clause is managed in two different phases of the pipeline:



Translation of C logic: Conditional clauses

The implementation of the conditional clause is managed in two different phases of the pipeline:



In this way **all the conditional branches are executed but the wrong ones measure zero**

Translation of C logic: Conditional Clauses

```
int main() {  
    int a = 4;  
    int b = a + 3;  
    int c = 0;  
    if (b > 0)  
    {  
        c = b * a;  
    }  
    return c;  
}
```

C code

```
builtin.module {  
  func.func @main() -> i32 {  
    %0 = arith.constant 4 : i32  
    %1 = iarith.addi_imm %0, 3 : i32  
    %2 = arith.constant 0 : i32  
    %3 = arith.constant 0 : i32  
    %4 = arith.cmpi sgt, %1, %3 : i32  
    cf.cond_br %4, ^0, ^1  
  ^0:  
    %5 = arith.muli %1, %0 : i32  
    func.return %5 : i32  
  ^1:  
    func.return %2 : i32  
  }
```

Classical IR

```
builtin.module {  
  func.func @main() -> i32 {  
    %q0 = quantum.init 4 : i32  
    %q1 = quantum.addi_imm %q0, 3 : i32  
    %q2 = quantum.init 0 : i32  
    %q3 = quantum.init 0 : i32  
    %q4 = quantum.cmpi %q1, %q3[4 : i64  
    %q5 = quantum.c_muli %q1, %q0, %q4  
    func.return %q5 : i32  
    %q7 = quantum.not %q4  
    func.return %q2 : i32  
  }
```

Quantum IR

Translation of C logic: Conditional Clauses

```
int main() {  
    int a = 4;  
    int b = a + 3;  
    int c = 0;  
    if (b > 0)  
    {  
        c = b * a;  
    }  
    return c;  
}
```

C code

```
builtin.module {  
    func.func @main() -> i32 {  
        %0 = arith.constant 4 : i32  
        %1 = iarith.addi_imm %0, 3 : i32  
        %2 = arith.constant 0 : i32  
        %3 = arith.constant 0 : i32  
        %4 = arith.cmpi sgt, %1, %3 : i32  
        cf.cond_br %4, ^0, ^1  
    ^0:  
        %5 = arith.muli %1, %0 : i32  
        func.return %5 : i32  
    ^1:  
        func.return %2 : i32  
    }  
}
```

Classical IR

```
builtin.module {  
    func.func @main() -> i32 {  
        %q0 = quantum.init 4 : i32  
        %q1 = quantum.addi_imm %q0, 3 : i32  
        %q2 = quantum.init 0 : i32  
        %q3 = quantum.init 0 : i32  
        %q4 = quantum.cmpi %q1, %q3[4 : i64  
        %q5 = quantum.c_muli %q1, %q0, %q4  
        func.return %q5 : i32  
        %q7 = quantum.not %q4  
        func.return %q2 : i32  
    }  
}
```

Quantum IR

Translation of C logic: Conditional Clauses

```
int main() {  
    int a = 4;  
    int b = a + 3;  
    int c = 0;  
    if (b > 0)  
    {  
        c = b * a;  
    }  
    return c;  
}
```

C code

```
builtin.module {  
    func.func @main() -> i32 {  
        %0 = arith.constant 4 : i32  
        %1 = iarith.addi_imm %0, 3 : i32  
        %2 = arith.constant 0 : i32  
        %3 = arith.constant 0 : i32  
        %4 = arith.cmpi sgt, %1, %3 : i32  
        cf.cond_br %4, ^0, ^1  
        ^0:  
        %5 = arith.muli %1, %0 : i32  
        func.return %5 : i32  
        ^1:  
        func.return %2 : i32  
    }  
}
```

Classical IR

```
builtin.module {  
    func.func @main() -> i32 {  
        %q0 = quantum.init 4 : i32  
        %q1 = quantum.addi_imm %q0, 3 : i32  
        %q2 = quantum.init 0 : i32  
        %q3 = quantum.init 0 : i32  
        %q4 = quantum.cmpi %q1, %q3[4 : i64  
        %q5 = quantum.c_muli %q1, %q0, %q4  
        func.return %q5 : i32  
        %q7 = quantum.not %q4  
        func.return %q2 : i32  
    }  
}
```

Quantum IR

Translation of C logic: Conditional Clauses

```
int main() {  
    int a = 4;  
    int b = a + 3;  
    int c = 0;  
    if (b > 0)  
    {  
        c = b * a;  
    }  
    return c;  
}
```

C code

```
builtin.module {  
    func.func @main() -> i32 {  
        %0 = arith.constant 4 : i32  
        %1 = iarith.addi_imm %0, 3 : i32  
        %2 = arith.constant 0 : i32  
        %3 = arith.constant 0 : i32  
        %4 = arith.cmpi sgt, %1, %3 : i32  
        cf.cond_br %4, ^0, ^1  
        ^0:  
        %5 = arith.muli %1, %0 : i32  
        func.return %5 : i32  
        ^1:  
        func.return %2 : i32  
    }  
}
```

Classical IR

```
builtin.module {  
    func.func @main() -> i32 {  
        %q0 = quantum.init 4 : i32  
        %q1 = quantum.addi_imm %q0, 3 : i32  
        %q2 = quantum.init 0 : i32  
        %q3 = quantum.init 0 : i32  
        %q4 = quantum.cmpi %q1, %q3[4 : i64  
        %q5 = quantum.c_muli %q1, %q0, %q4  
        func.return %q5 : i32  
        %q7 = quantum.not %q4  
        func.return %q2 : i32  
    }  
}
```

Quantum IR

Translation of C logic: Conditional Loops

Loops are implemented in the quantum IR by unrolling them up to a **maximum number of iterations** (max_iter), rewriting them as a series of **nested if clauses**. This approach supports both loops with **statically known and dynamically unknown bounds** at compile time.

max_iter = 3

```
int main() {  
    int a = 2;  
    int b = 0;  
    for (int i = 0; i < a; i = i + 1)  
    {  
        b = b + 1;  
    }  
    return b;  
}
```

```
builtin.module {  
  func.func @main() -> i32 {  
    %0 = arith.constant 2 : i32  
    %1 = arith.constant 0 : i32  
    %2 = arith.constant 0 : i32  
    %3 = arith.cmpi slt, %2, %0 : i32  
    cf.cond_br %3, ^0, ^1  
  ^0:  
    %4 = iarith.addi_imm %1, 1 : i32  
    %5 = iarith.addi_imm %2, 1 : i32  
    %6 = arith.cmpi slt, %5, %0 : i32  
    cf.cond_br %6, ^2, ^3  
  ^1:  
    func.return %1 : i32  
  ^2:  
    %7 = iarith.addi_imm %4, 1 : i32  
    %8 = iarith.addi_imm %5, 1 : i32  
    %9 = arith.cmpi slt, %8, %0 : i32  
    cf.cond_br %9, ^4, ^5  
  ^3:  
    func.return %4 : i32  
  ^4:  
    %10 = iarith.addi_imm %7, 1 : i32  
    %11 = iarith.addi_imm %8, 1 : i32  
    %12 = arith.cmpi slt, %11, %0 : i32  
    cf.cond_br %12, ^6, ^7  
  ^5:  
    func.return %7 : i32  
  ^6:  
  ^7:  
    func.return %10 : i32  
  }  
}
```

Translation of C logic: Conditional Loops

Loops are implemented in the quantum IR by unrolling them up to a **maximum number of iterations** (`max_iter`), rewriting them as a series of **nested if clauses**. This approach supports both loops with **statically known** and **dynamically unknown bounds** at compile time.

```
int main() {  
    int a = 2;  
    int b = 0;  
    for (int i = 0; i < a; i = i + 1)  
    {  
        b = b + 1;  
    }  
    return b;  
}
```

max_iter = 3

```
builtin.module {  
  func.func @main() -> i32 {  
    %0 = arith.constant 2 : i32  
    %1 = arith.constant 0 : i32  
    %2 = arith.constant 0 : i32  
    %3 = arith.cmpi slt, %2, %0 : i32  
    cf.cond_br %3, ^0, ^1  
1  ^0:  
    %4 = iarith.addi_imm %1, 1 : i32  
    %5 = iarith.addi_imm %2, 1 : i32  
    %6 = arith.cmpi slt, %5, %0 : i32  
    cf.cond_br %6, ^2, ^3  
2  ^2:  
    %7 = iarith.addi_imm %4, 1 : i32  
    %8 = iarith.addi_imm %5, 1 : i32  
    %9 = arith.cmpi slt, %8, %0 : i32  
    cf.cond_br %9, ^4, ^5  
3  ^4:  
    %10 = iarith.addi_imm %7, 1 : i32  
    %11 = iarith.addi_imm %8, 1 : i32  
    %12 = arith.cmpi slt, %11, %0 : i32  
    cf.cond_br %12, ^6, ^7  
4  ^6: !!!  
    ^7:  
    func.return %10 : i32  
  }  
}
```



Translation of C logic: Conditional Loops

```
builtin.module {  
  func.func @main() -> i32 {  
    %q0 = quantum.init 2 : i32  
    %q1 = quantum.init 0 : i32  
    %q2 = quantum.init 0 : i32  
    %q3 = quantum.cmpi %q2, %q0[2 : i64] : i32  
    %q4 = quantum.c_addi_imm %q1, 1 : i32, %q3 : i32  
    %q5 = quantum.c_addi_imm %q2, 1 : i32, %q3 : i32  
    %q6 = quantum.cmpi %q5, %q0[2 : i64] : i32  
    %q8 = quantum.and %q3, %q6  
    %q7 = quantum.c_addi_imm %q4, 1 : i32, %q8 : i32  
    %q10 = quantum.and %q3, %q6  
    %q9 = quantum.c_addi_imm %q5, 1 : i32, %q10 : i32  
    %q11 = quantum.cmpi %q9, %q0[2 : i64] : i32  
    %q13 = quantum.and %q3, %q6
```

```
    %q14 = quantum.and %q13, %q11  
    %q12 = quantum.c_addi_imm %q7, 1 : i32, %q14 : i32  
    %q16 = quantum.and %q3, %q6  
    %q17 = quantum.and %q16, %q11  
    %q15 = quantum.c_addi_imm %q9, 1 : i32, %q17 : i32  
    %q18 = quantum.cmpi %q15, %q0[2 : i64] : i32  
    %q19 = quantum.not %q18  
    func.return %q12 : i32  
    %q20 = quantum.not %q11  
    func.return %q7 : i32  
    %q21 = quantum.not %q6  
    func.return %q4 : i32  
    %q23 = quantum.not %q3  
    func.return %q1 : i32
```

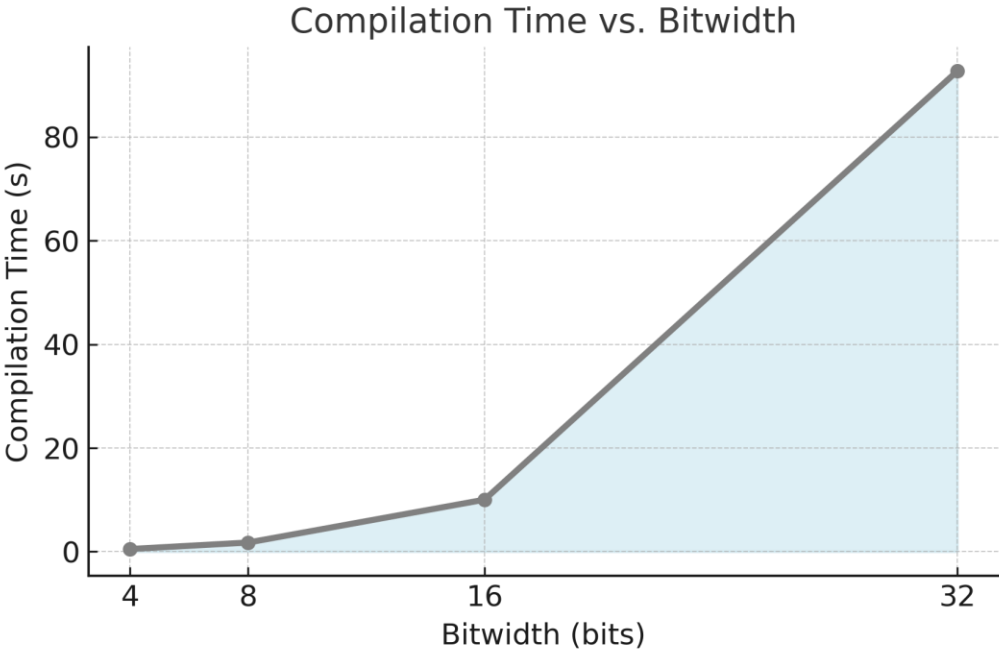
```
}
```

```
}
```



Time Benchmark: Bitwidth vs. Compilation Cost

- This analysis investigates how increasing integer bitwidth affects quantum compilation time, highlighting the tradeoff between arithmetic precision and synthesis feasibility.
- The benchmark program includes six statically unrolled iterations with chained multiplications and additions, stressing the quantum transformation pipeline.
- These results underline a **key tradeoff** in quantum compilation: **precision-rich logic demands careful management of performance overhead.**
- Compilation time grows superlinearly with bitwidth due to the $O(n^2)$ cost of QFT-based arithmetic and increased control depth.
- Note that the Compilation Time difference is only due to Backend Lowering to Quantum Assembly Language (QASM), **the complexity of Intermediate Representation does not depend on the bitwidth.**



Bitwidth (bits)	Compilation Time (s)
4.0	0.5
8.0	1.75
16.0	10.02
32.0	92.73



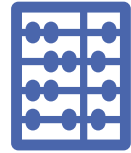
Quantum Complexity

- All circuits have been decomposed into the Clifford+T universal gate set to enable backend-independent complexity analysis and fair cost estimation across fault-tolerant platforms.
- Due to static single assignment (SSA) and lack of in-place operations, qubit counts grow proportionally with control depth and register duplication across branches.
- The complexity trends observed confirm that circuit size correlates not only with program size, but more strongly with arithmetic intensity and register bitwidth, consistent with the previous timing benchmarks.

#Test	num_qubits	total_gates	CX	T
01	48	563	0	0
02	1037	43497	11268	378
03	74	1178	19	0
04	80	10839	4512	0
05	1037	57136	17338	378
06	74	1178	19	0
07	64	1583	121	0
08	1632	51157	14678	1441
09	80	10839	4512	0



Number of Qubits: Total number of logical qubits used in the final circuit.



Total Gate Count: Overall number of quantum gates in the synthesized circuit.



CX Gates: Number of controlled-NOT (CNOT) gates, which are crucial indicators of circuit depth and error sensitivity on real hardware.



T Gates: Number of T gates, representing the non-Clifford contribution to the overall complexity and a key metric in fault-tolerant quantum computing.



Chained Arithmetic and No-Cloning

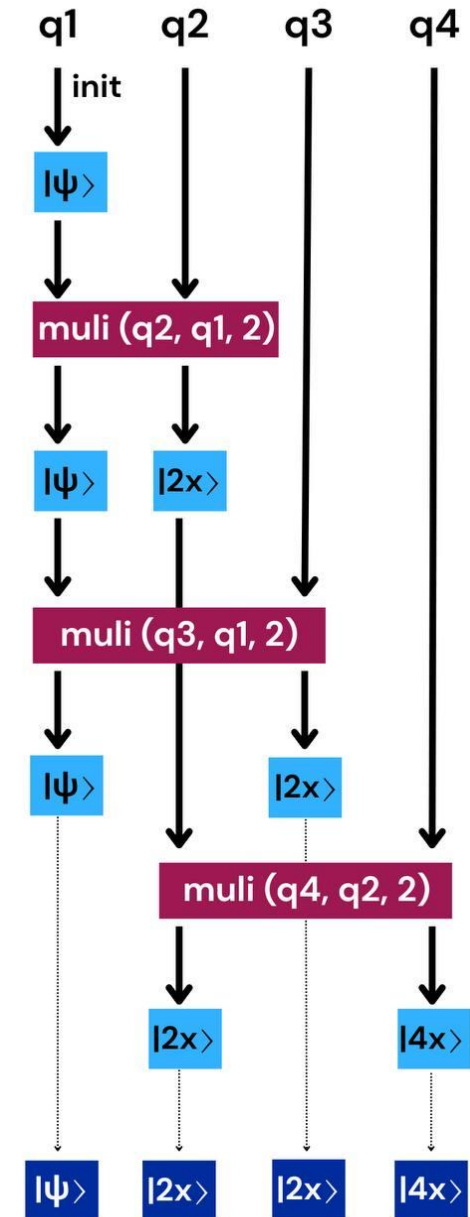
$$|\psi\rangle_{q_1} = \sum_x \alpha_x |x\rangle$$

From the example:

$$|\phi\rangle_{q_1, q_2, q_3, q_4} = \sum_x \alpha_x |x\rangle \otimes |2x\rangle \otimes |2x\rangle \otimes |4x\rangle$$

In general final state has form:

$$\sum_x \alpha_x |x\rangle \otimes |f_1(x)\rangle \otimes |f_2(x)\rangle \otimes \dots$$





Chained Arithmetic and No-Cloning

Classical Case: No Superposition

When qubits encode definite classical values, each output is deterministically derived and no entanglement arises.

Quantum Case: Superposition and Entanglement

Applying arithmetic operations to superposed inputs produces entangled outputs; registers are no longer independent.

Collapse on Measurement

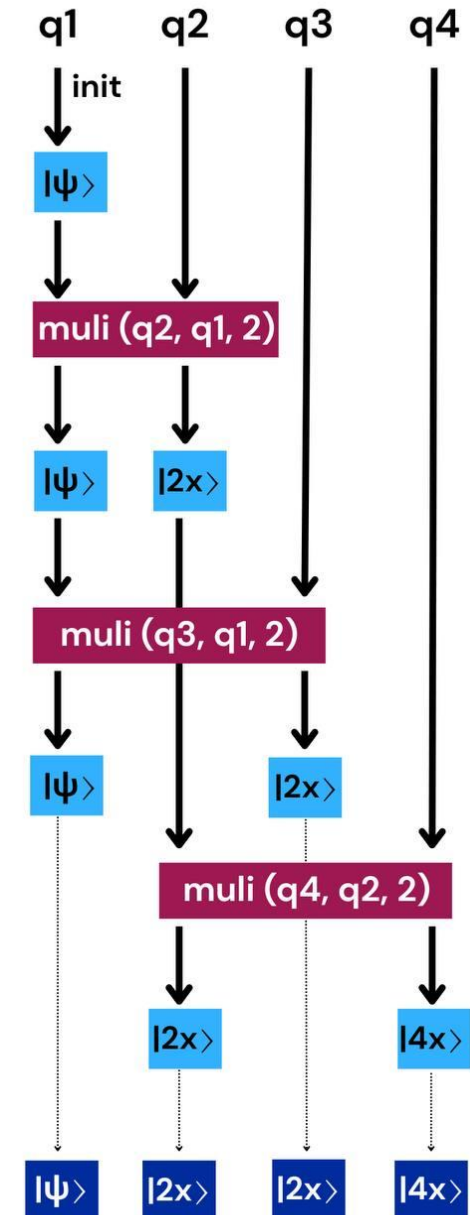
Measuring one output register collapses all correlated registers due to entanglement, preventing reuse of partial results.

No Advantage from Postselection

Even using Grover's algorithm to amplify a desired input state, the outputs remain tied to the same classical function, with no gain over a classical path.

Core Challenge: Meaningful Superposition Use

The difficulty lies not in implementing arithmetic quantumly, but in leveraging superposition for computation that cannot be classically reproduced.





Conclusions

Modular Quantum Compilation

We introduced a **modular pipeline** for synthesizing quantum circuits from high-level C programs.

The approach is based on **intermediate representations (IRs)** that preserve semantics and support quantum constraints like **unitarity**, **reversibility**, and **SSA form**.

Key Advantages:

- **IR-centric architecture** enables semantic clarity, structural reuse, and introspection.
- **No early commitment** to fixed transformations: abstraction levels can be revisited.
- Facilitates **debugging**, **optimization**, and **resource-aware synthesis**.



Future Developments

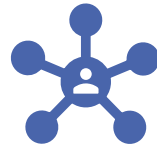


Bidirectional and Flexible Workflows

Support for **lifting** QASM to quantum IR and reverse mapping to classical control logic

Enable **round-trip transformations** for debugging and IR-level optimization

Integrate alternative frontends (e.g., Verilog) for shared IR reuse



Dialect Interoperability

Align with existing IRs like **QIRO**, **xDSL**, and **MLIR**

Define translation layers between dialects

Contribute toward **shared community standards** for quantum IRs



Target-Aware Gate Synthesis

Support configurable native gate sets per backend

Allow **architecture-specific lowering** (e.g., Toffoli, MCRz)

Enable trade-offs between **abstraction** and **performance**



IR-Level Benchmarking

Compare multiple optimization stacks on:

Qubit usage

Gate count and depth

Fidelity

Facilitate reproducible experiments across toolchains

**Thank you
for the
attention**

